

PeTTa-Memory Integration into Iter — Revised Design (v2)

Date: 2026-09-03 · **Status:** draft for review · **Supersedes:** petta_memory_deployment_plan v1 (P2 iter.py hook withdrawn per verdict @00:21)

Ground truth (re-verified @00:30)

- petta-memory repo agent/parser-validation @ b48975d, clean, pushed; **731 tests + 206 subtests PASS (51.7s)**.
- Journal: memory/_journal/journal.metta, 71 clusters (46E/16F/9P), 0 orphan/dangling links.
- Runtime: patham9/iter @ f4064d9, unmodified. All integration is via transformations/ + tools/ globbed from disk per step — **no iter.py edits, no fork, no restart, rollback = rename file with leading _**.
- Benchmark (fresh process, median of 7): import 0.06s; load 71c journal **0.59s**; load synthetic 600c journal **4.47s** → exceeds DYNAMIC_TIMEOUT=5s at ~650c. **Prerendered index is mandatory, not optional.**

Architecture

All PeTTa coupling lives in the **writer**, which runs rarely (burst boundaries, explicit appends). Readers are pure file reads. No per-step process ever imports petta_memory or parses the journal.

Component 1 — _petta_journal.py (transformation, writer)

Runs every step like all transformations (failure-isolated by iter's wrapper, 5s budget), but is cheap by construction:

1. **Fast path (every step):** stat journal + index; do nothing unless a burst boundary is detected. Cost: microseconds.
2. **Burst boundary** (iter's nop → new _burst marker): invoke the model to write **one Episode per burst** (not thin per-step tool logs — those duplicate experience.json). Episode = model-written summary of the burst: goal, key actions, outcome, salient quotes. Append via append_cluster (atomic-validating API, never raw file writes).
3. **Idempotency:** keyed by the last tool_call_id of the journaled step — transformations rerun on no-tool-call retries; already-journaled IDs are skipped, so crashes backfill instead of duplicating.
4. **Index maintenance:** after every append, re-render memory/_journal/_index.md (prerendered view, see below). Amortized: only on writes.
5. **Stamps:** store both UTC ISO stamp and iter's local step string (UTC+7) on every cluster.

Component 2 — `_petta_context.py` (transformation, reader)

1. Reads `_index.md` only — **no petta import, no journal parse**. Target <50ms/step, comfortably inside 5s even on slow disk.
2. Injects it as a **bracketed user message** (iter notice style), tail-placed for KV-cache reuse.
3. **Index content = the Belief layer only**: current Beliefs + Promotions with Supersedes fully resolved (stale facts never rendered). Size-capped (~1500c, newest-first tail). Raw Episodes are **never** injected — recall-tool only. This closes the prompt-injection hole: group-chat input persisted raw and auto-injected into every future context is a standing attack channel; agent-authored, supersession-resolved Beliefs are not.
4. **System of record**: Belief/Promotion layer is the curated record the model sees; `memory/*.txt` stays iter's own curated LTM (3000c cap); raw journal is the evidence store behind both.

Component 3 — `petta_append` / `petta_recall` (tools)

- `petta_append(type, payload)`: wraps `append_cluster` for Facts/Beliefs/Evidence/links (Supersedes, EvidenceFor). Fixes the observed failure mode of manual file edits (3 historical Supersedes corrections went through edits; one silently failed to persist).
- `petta_recall(query, k)`: PeTTa query over the full journal incl. raw Episodes; returns compact results. Explicit `limit` always passed (`query_about` default is 20).
- Both keep imports lazy inside `run()`; both run under iter's subprocess timeout.

Answers to review questions

- **Q1 (Episode granularity)**: per-burst, model-written, at `nop→new_burst`. Makes compaction nearly moot.
- **Q2 (ranking)**: don't rank Episodes; curate. Inject Belief layer only.
- **Q3 (compaction)**: deferred — per-burst Episodes grow slowly; revisit at index-cap pressure.
- **Q4 (supersession)**: keep Supersedes-only, **provided every rendered view resolves it** so the model never sees a stale fact. Enforced in the index renderer + `petta_recall` output filter.

Evidence-count caveat (review point 7)

The E2E STV 0.049/0.855 → 0.004/**0.9986** result likely double-counts correlated items (revision over ≤63 shared-evidence clusters) — exactly what ωPLN evidence fibering is meant to rule out. Before citing confidence gains, report the **independent evidence count**, not the cluster count. Action: when integrating with GOAL-SYSTEM, compute fibering-aware evidence counts and re-run `solve_incident`; treat 0.9986 as unvalidated.

GoalChainer note: the union-resolved merge in `metta_reasoner.py` with 4 red tests should go to a **branch for Patrick**, not origin/main.

Rollback & safety

- Every component is a file in `transformations/` or `tools/`; disable = rename with leading `_` (live, no restart).
- Writer failure → `[RUNTIME ERROR ...]` message, never a crash (iter's wrapper).
- Journal is append-only through a validating API; existing audit (0 orphans) is reproducible.

Phasing

- **P2 (revised):** `_petta_journal.py` writer + index renderer + per-burst Episodes. Bench: index render <1s at 600c.
- **P3 (revised):** `_petta_context.py` reader + bracketed-user injection. Bench: <50ms/step.
- **P5 (new):** `petta_append` / `petta_recall` tools.
- **P6 (new):** evidence-fibering count validation before any confidence claims; GC merge → branch for Patrick.